SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA0421 **COURSE NAME:** Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments. (BL2, BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Total Marks:** 25

Annexure A

Concept Mapping

Code of Conduct:

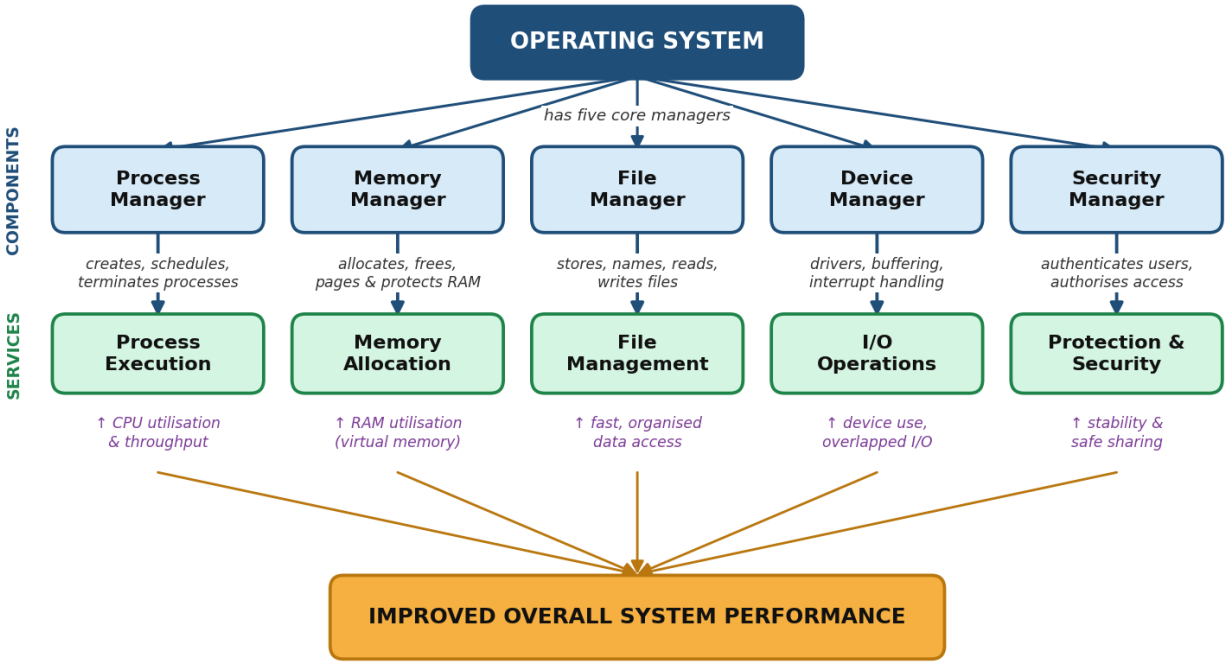
*I **Saai Vardhan D (192525205)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Saai Vardhan D

Question 1 – Concept Mapping: OS Components and Services [10 marks]

An Operating System sits between the user and the hardware and offers services to programs [1]. Each core component (manager) of the OS exists to deliver one of those services. The concept map shows the hierarchy: **OS → components → services → performance**.



1. Mapping of components to services and their relationship

Component	Mapped service	Relationship (how the component delivers the service)	Windows example [4]
Process Manager	Process Execution	Creates and terminates processes and threads, schedules the CPU among them and handles synchronisation, so that programs actually run.	Process & Thread Manager, scheduler
Memory Manager	Memory Allocation	Allocates and frees RAM, keeps track of which parts are in use, and uses paging/virtual memory so each process gets its own protected address space.	Virtual Memory Manager (VMM)
File Manager	File Management	Creates, names, stores, reads, writes and deletes files and directories, and tracks free space and file attributes on secondary storage.	NTFS file system driver
Device Manager	I/O Operations	Controls hardware through device drivers and uses buffering, caching, spooling and interrupts to hide device details from programs.	I/O Manager, Plug and Play Manager
Security Manager	Protection & Security	Authenticates users and checks permissions on files, memory and devices, so that one process or user cannot interfere with another.	Security Reference Monitor

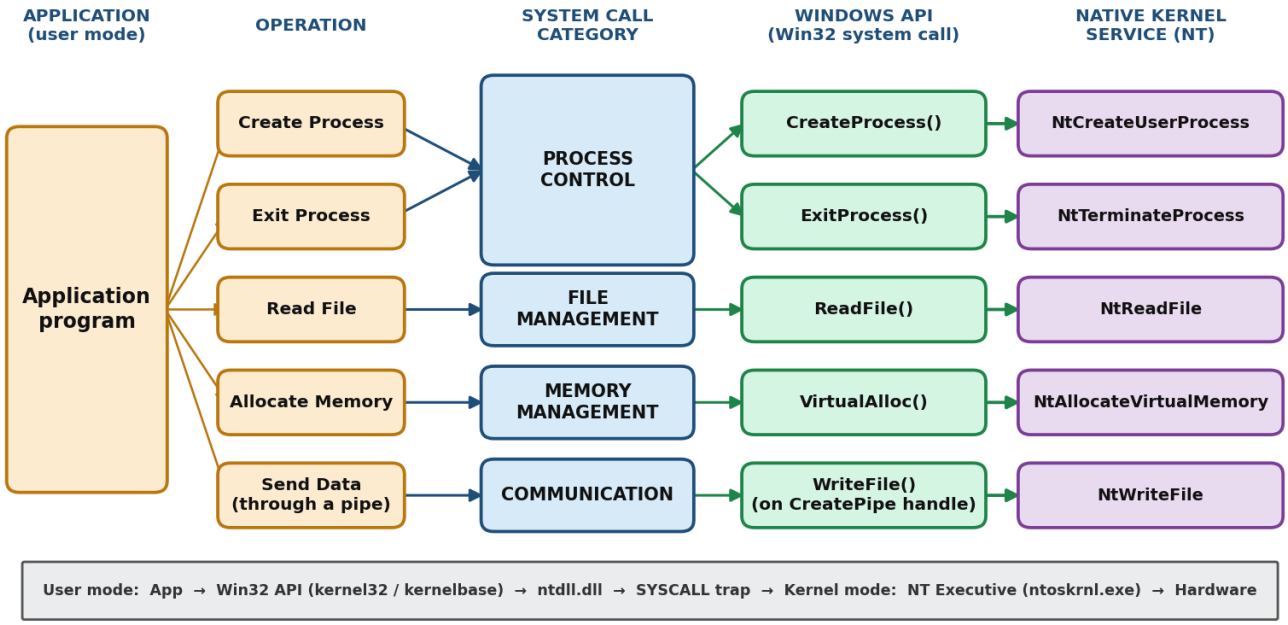
2. How the components collectively improve system performance

Performance aspect	Contribution of the components
CPU utilisation & throughput	The Process Manager keeps several processes ready (multiprogramming) and switches the CPU whenever one waits for I/O, so the CPU is never idle [1].
Memory utilisation	The Memory Manager's virtual memory lets more programs run than physical RAM could hold, and protects each address space [2].
Speed of data access	The File Manager's caching and organised directories make storage access fast, with the Device Manager's drivers doing the actual reads and writes.
Device utilisation	Buffering and spooling in the Device Manager overlap slow I/O with computation, so CPU and devices work in parallel.
Reliability & stability	The Security Manager's protection stops faulty or malicious programs from corrupting others, so the whole system stays stable and shareable.

Working together – example: when a user opens a document, the Process Manager schedules the application, the Memory Manager gives it an address space, the Security Manager checks the user's permission, the File Manager locates the file and the Device Manager reads it from disk through the driver. Because every component handles its own task, **no single part becomes a bottleneck, and resources are used efficiently and safely.**

Question 2 – Concept Mapping: System Calls and OS Services [10 marks]

A system call is the interface through which a program asks the OS kernel for a service [1]. System calls fall into categories such as process control, file management, memory management and communication [3]. On Windows, programs normally call the **Win32 API**, which in turn invokes the native NT kernel service.



1. Mapping of operations to system call categories, with Windows system calls

Operation	Category	Reason	Windows system call [5]	Native service [4]
Create Process	Process Control	Starts a new process with its own address space	CreateProcess()	NtCreateUserProcess
Read File	File Management	Transfers data from a file on disk into a buffer	ReadFile()	NtReadFile
Allocate Memory	Memory Management	Reserves and commits pages of virtual memory	VirtualAlloc()	NtAllocateVirtualMemory
Send Data (pipe)	Communication	Passes data between processes through a pipe (created with CreatePipe)	WriteFile() on the pipe handle	NtWriteFile
Exit Process	Process Control	Ends the process and releases its resources	ExitProcess()	NtTerminateProcess

Note: "Process Control" appears twice because both creating and terminating a process are process-control operations.

2. The five operations in one program (Windows C code)

```
CreateProcess(NULL, "child.exe", NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi);    // Process Control
HANDLE hF = CreateFile("data.txt", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, NULL);
ReadFile(hF, buffer, 512, &bytesRead, NULL);                                    // File Management
void *p = VirtualAlloc(NULL, 4096, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);   // Memory Management
CreatePipe(&hRead, &hWrite, &sa, 0);                                            // set up the pipe
WriteFile(hWrite, buffer, bytesRead, &written, NULL);                           // Communication
ExitProcess(0);                                                                 // Process Control
```

3. How a system call works

The program calls a Win32 function, which passes through ntdll.dll and executes a SYSCALL instruction. The CPU then switches from user mode to kernel mode, the NT Executive performs the service, and control returns with the result [1][4]. This keeps hardware and memory protected while still giving programs controlled access to OS services. (Linux equivalents: fork(), read(), mmap(), pipe()/write(), exit().)